

Experience is the Best Teacher: Augmenting LLM Reasoning with Knowledge Learned from the Past

Mingjun Zhou
National University of Defense
Technology
National Key Laboratory of Big Data
and Decision
Changsha, Hunan, China
zhoumingjun24@nudt.edu.cn

Weixin Zeng*
National University of Defense
Technology
National Key Laboratory of Big Data
and Decision
Changsha, Hunan, China
zengweixin13@nudt.edu.cn

Xiang Zhao*
National University of Defense
Technology
National Key Laboratory of Big Data
and Decision
Changsha, Hunan, China
xiangzhao@nudt.edu.cn

Abstract

Large Language Models (LLMs), despite of their success in complex reasoning, approach each problem instance separately, generating solutions from scratch. To fill in this gap, researchers propose to store the past experiences in an episodic memory, allowing LLM to reuse these gradually accumulated memories at inference time, achieving the transition from isolated inference toward experience-aware language agents. Nevertheless, the memories stored by these approaches are mainly specific “question-answer” pairs that lack generalizability, and the memories are retrieved mainly based on surface form similarity during reasoning, often failing to provide experiences with genuinely analogous reasoning structures. To address these limitations, we introduce Tricog, a novel framework grounded in cognitive structure migration theory, aiming to augment LLM reasoning with knowledge summarized from past experiences. Specifically, we build a knowledge repository of *generalizable schemas*, each distilled from the LLM’s complete self-reflection on a past instance - including its solution, judgment, and correction - and associated with its source problem’s *core reasoning structure*. Tricog is also equipped with a highly effective and discriminating knowledge retriever, which matches a new problem’s reasoning structure against those in the knowledge repository to retrieve schemas that are highly relevant and also diverse, providing robust guidance for the LLM’s reasoning process. Extensive experiments on existing benchmarks demonstrate that Tricog achieves superior performance against state-of-the-art solutions, validating the effectiveness of our proposal.

CCS Concepts

• **Information systems** → *Question answering*.

Keywords

Large Language Model; Memory-Augmented Generation; Self-Learning

ACM Reference Format:

Mingjun Zhou, Weixin Zeng, and Xiang Zhao. 2026. Experience is the Best Teacher: Augmenting LLM Reasoning with Knowledge Learned from the

Past. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792630>

1 INTRODUCTION

Recent advances in Large Language Models (LLMs) have demonstrated their remarkable capabilities in complex reasoning and problem-solving [7, 11, 35]. However, once deployed, their parameters are largely fixed, and it is difficult for them make use of the past experiences (e.g., questions, successes, or mistakes) during inference due to the huge computational costs of training and fine-tuning [4, 36]. They tackle each problem instance in isolation and reason from first principles [15, 31], often re-deriving the same insights and re-committing the same errors [24].

Drawing inspiration from human cognition, where past experiences are constantly internalized into durable, reusable memories, a promising paradigm is to dynamically curate a compact library of *memories* (e.g., reusable strategies [24], solution sketches [27], and code snippets [12]) from LLMs’ own operational history and *augment LLMs with such memories* that extend beyond their frozen parameters during new inference [28, 33]. Specifically, existing approaches operate by building an evolving repository of past problem-solving experiences. When faced with a new problem, these models retrieve relevant memories from the repository, which are then used as in-context examples to guide LLM’s reasoning, thereby enabling it to learn from practice [6, 12, 24, 25, 27, 30, 34, 38].

While this paradigm of learning from experience represents a significant step forward, notable issues persist in its current implementations. First, *existing methods are limited in constructing their memory contents*. As illustrated in Figure 1(a), the structure of their memory repositories is overly simplistic, with contents confined to previous “question-answer (QA)” pairs [12, 24]. However, these QA pairs, in essence, lack generalizability, which may not effectively apply to novel questions. More importantly, in addition to the plain QA pairs, LLM can actually acquire much more from past experiences by self-evaluating the quality of the answers and providing potential correction directions, which are important pieces of knowledge that can be referred to when new questions arise. Additionally, *these methods do not make full use of the memory*. They retrieve relevant memory contents based on superficial semantic similarity between the new query and past experiences [12, 24, 27, 30], which, however, may retrieve problems with similar surface features but fundamentally different underlying reasoning structures or solution paradigms. For example, two math problems

*Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792630>

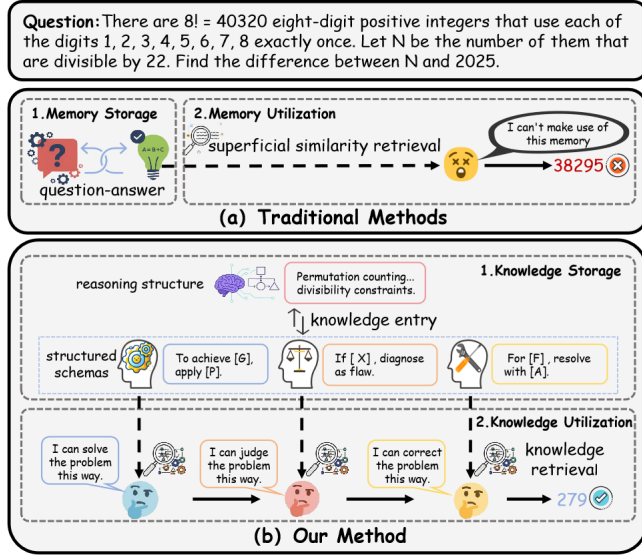


Figure 1: A conceptual comparison illustrating our core contributions. (a) Traditional methods store simplistic question-answer pairs and utilize them via superficial similarity retrieval, a process that is unreliable and error-prone. (b) Our framework conceptualizes experience as structured knowledge. A core reasoning structure enables precise retrieval, while a set of structured schemas guides a robust, multi-stage reasoning process of solving, judging, and correcting.

might both involve “trains” and “speed”, yet one may require solving a system of linear equations, while the other is a simple unit conversion. In such cases, retrieving based on surface form similarity may introduce significant noise, even misleading the LLM from making the correct inference [14].

To address the aforementioned issues, in this work, we *move beyond the simple storage of past experiences as memories and propose to transform these memories into an organized repository of knowledge to better guide LLM reasoning*. Our proposal, Tricog, is grounded in *cognitive structure migration theory* [3], which posits that all meaningful learning necessarily involves migration that is built upon the original *cognitive structure*, i.e., the content and organization of the internal knowledge system of a student. The cognitive structure is expected to be generalizable, so that the knowledge can be easily transferred to new cases, and also discriminating, so that the most appropriate knowledge can be retrieved during inference. As a deployed LLM does not possess such an intrinsic cognitive structure that can continuously evolve with experience, we construct and maintain an external repository of knowledge, which is extracted and organized from the LLM’s operational history, to serve as the cognitive structure and guide LLM’s reasoning.

Regarding the construction of the knowledge repository, we operationalize the concept of cognitive structure by transforming raw problem-solving instances into knowledge that is both *generalizable* and *discriminating*. To achieve generalizability, we first guide LLM through a deep self-reflection cycle: it first solves a problem, then judges the quality of its own answer, and proposes corrections.

Through *schema generalization*, this complete reflective trace is distilled into a set of structured schemas for each stage, i.e., solving, judging, and correcting schemas. By design, these schemas strip away superficial, instance-specific details to crystallize the underlying reasoning structures, evaluation criteria, and remedial strategies. To ensure the knowledge is also discriminating, we create a unique key for each knowledge entry through *structural abstraction* that extracts the core reasoning structure of the corresponding problem, which moves beyond superficial semantics and serves as a highly discriminating index for retrieval. *As to the effective utilization of the knowledge repository during inference*, we design a diversity-aware knowledge retrieval mechanism. Specifically, by comparing the reasoning structure of the new problem to the index in the knowledge repository, a set of highly relevant knowledge entries are first retrieved, which are then refined by considering both the relevance and the diversity, thereby producing a more robust context for LLM reasoning. Extensive experiments demonstrate that Tricog achieves state-of-the-art performance on popular complex reasoning benchmarks, validating the effectiveness of our design. In summary, the main contributions are summarized as follows:

- We propose Tricog, a self-learning framework rooted in *cognitive structure migration theory* that augments LLM reasoning with knowledge summarized from past experiences. It constructs and maintains an external, evolving cognitive structure, enabling LLM to learn from a complete reflection cycle (i.e., solving, judging, and correcting) of experiences, thereby gradually improving the reasoning ability.
- The evolving knowledge repository is constructed by considering both the generalizability and discriminability of the knowledge summarized from past experiences. Generalizability is achieved via a schema generalization process that distills LLM’s full self-reflection experience into reusable, structured schemas. Discriminability is ensured by a parallel structural abstraction process that extracts a problem’s core reasoning structure that serve as precise index.
- To avoid the pitfalls of retrieval based on superficial similarity, we design a novel, diversity-aware knowledge retrieval mechanism that can effectively identify a set of relevant and strategically diverse schemas to better inform LLM reasoning.

2 RELATED WORKS

Memory-Augmented Generation and Reasoning. A prominent line of research equips LLMs with external memory to enable learning from past experiences. These approaches typically focus on learning from interactions, initially from correcting mistakes. For instance, MemPrompt [18] captures user feedback on errors to prevent the model from repeating similar mistakes in future interactions. This principle was later extended to reusing successful solution components. Thought-Retriever [9], for example, logs the model’s chain-of-thought from past queries for reuse, while Dynamic Cheatsheet (DC) [24] enables models to store and apply accumulated strategies and code snippets at inference time. The multi-agent framework Xolver [12] equips an LLM with a persistent memory that integrates diverse modalities such as tool use and collaborative interactions. These methods generally rely on retrieval

mechanisms based on vector-embedding semantic similarity. Our work differs from these approaches in the structure and representation of knowledge. Instead of merely storing interaction snippets, we distill an LLM’s full self-reflection cycle—encompassing solving, judging, and correcting—into explicit, structured schemas. More importantly, we introduce a retrieval mechanism that matches the core reasoning structure of problems, aiming for a more precise and principled knowledge access than is typical in prior methods.

Retrieval-Augmented Generation. Retrieval-Augmented Generation (RAG) is a paradigm designed to enhance LLMs by retrieving information from external knowledge sources. The primary goal of RAG is to improve the factual accuracy of model outputs and mitigate hallucination [4, 36, 37]. A typical RAG workflow involves two stages: first, a retriever, often based on vector similarity, identifies the top- k relevant text passages from a large, typically static corpus of documents based on a user’s query. Second, these retrieved passages are prepended to the original query as context for the LLM to generate the final answer [15, 31]. RAG has proven highly effective for open-domain question answering and tasks requiring precise factual support. While our framework incorporates retrieval, its objective and the nature of its knowledge source differ fundamentally from traditional RAG. Whereas RAG typically consults a fixed, external knowledge base for factual information, we construct a dynamic repository from the model’s own evolving operational history. Our goal is thus not to retrieve isolated facts, but to improve the model’s core reasoning abilities by reusing past, structured problem-solving experiences.

Test-Time Learning. Test-time learning, also referred to as online or incremental learning, encompasses methods where a model learns during inference without conventional offline finetuning [21]. A significant branch within this paradigm is intra-task refinement. Frameworks like Reflexion [23] and Self-refine [19] enable models to iteratively improve their outputs on a given task through self-generated feedback, though this learning is typically transient and does not persist to subsequent, independent problems. Another branch focuses on building a durable, cross-task memory. Works such as Xolver [12] and DC [24] archive successful problem-solving experiences into an evolving repository, allowing the model to benefit from its accumulated knowledge. Our work aligns with the durable memory branch. Our contribution is that we distill the model’s full self-reflection cycle into a more comprehensive, structured form of knowledge, rather than only archiving successful solutions. Furthermore, our retrieval is based on a problem’s core reasoning structure, aiming for more precise knowledge access than the semantic similarity matching common in prior methods.

3 METHODOLOGY

To enable an LLM to learn from its past experiences, we introduce Tricog, a framework that achieves continuous self-improvement by constructing and leveraging an external knowledge repository. The core idea is to transform the stateless, isolated problem-solving approach into a dynamic learning process that accumulates and transfers knowledge.

3.1 Framework Overview

The overall architecture of Tricog, illustrated in Figure 2, operates as a closed-loop system. It synergistically integrates two key processes: (1) Knowledge Repository Construction, which distills problem-solving experiences into structured knowledge, and (2) Knowledge Retrieval, which identifies and provides relevant historical knowledge for new problems. These processes together empower a central knowledge-augmented reasoning loop. Through this cycle of construction, retrieval, and reasoning, Tricog continuously learns from experience to enhance its problem-solving capabilities.

3.2 Foundations of Knowledge

Existing methods, which often store experiences as simple “question-answer (QA)” pairs, is not generalizable and overlook valuable procedural information. An LLM can acquire much more from an experience by self-evaluating the quality of its solution and proposing corrections. Therefore, we propose to extract *reusable knowledge* from past experiences. Our framework’s design is grounded in *cognitive structure migration theory* [3] and aims to construct an effective external cognitive structure for the LLM. An ideal cognitive structure should possess two key properties: *generalizability*, to ensure knowledge can be transferred to new situations, and *discriminability*, to ensure the most appropriate knowledge is retrieved when needed.

More specifically, we guide the LLM through a self-reflection cycle, which consists of solve, judge, and correct phases. Given a complex reasoning problem q , the process commences with the solving phase to produce an initial solution (T_0, R_0) , comprising a reasoning trace and a candidate answer:

$$(T_0, R_0) \leftarrow \mathcal{M}(p_{\text{solve}} \parallel q), \quad (1)$$

where $\parallel$ denotes string concatenation.

Subsequently, the model enters the judgment phase to critically review a given solution (T_i, R_i) and produces a structured feedback tuple $S_i = (F_i, s_i)$. In this tuple, F_i is a critique in natural language, and $s_i \in \{0, 1\}$ is a binary quality score:

$$S_i \leftarrow \mathcal{M}(p_{\text{judge}} \parallel q \parallel (T_i, R_i)). \quad (2)$$

If a solution is deemed flawed ($s_i = 0$), the model proceeds to an iterative correction phase, leveraging the problem q , the flawed solution (T_i, R_i) , and its corresponding feedback S_i to generate a refined solution (T_{i+1}, R_{i+1}) :

$$(T_{i+1}, R_{i+1}) \leftarrow \mathcal{M}(p_{\text{correct}} \parallel q \parallel (T_i, R_i) \parallel S_i). \quad (3)$$

This cycle of judgment and correction repeats until an acceptable solution is produced (i.e., $s_i = 1$) or a predefined maximum number of attempts is reached. The record of this entire cycle constitutes the complete reflective trace, which serves as the raw material for our structured knowledge.

To transform this instance-specific trace into the desired cognitive structure, it is abstracted into two core components. To ensure discriminability, the problem’s core logic is extracted as a reasoning *structure*, denoted as q_{abs} . To provide generalizability, the reusable strategies within the trace are distilled into a set of *schemas*, $\{S_{\text{solve}}, S_{\text{judge}}, S_{\text{correct}}\}$. Together, they form a single knowledge entry, $(q_{\text{abs}}, \{S_{\text{solve}}, S_{\text{judge}}, S_{\text{correct}}\})$. The details are elaborated in the following sections.

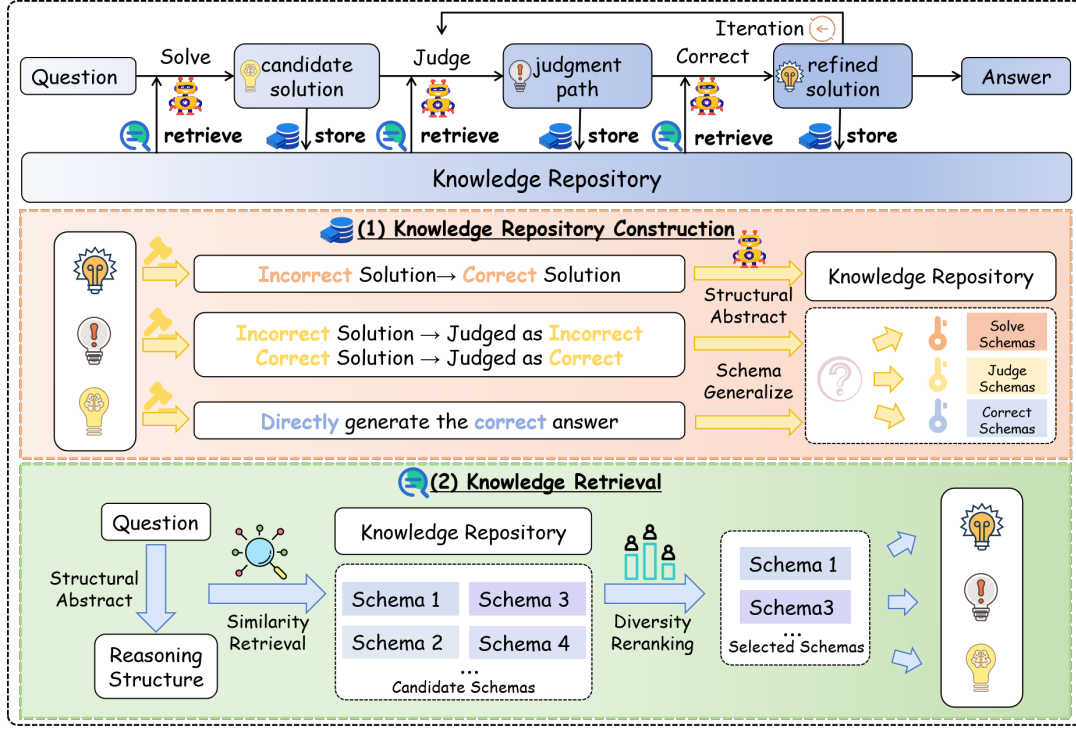


Figure 2: An overview of our proposed Tricog framework. Top: The core *knowledge-augmented reasoning loop*, where a solution is progressively refined through solve, judge, and correct phases, continuously interacting with the knowledge repository. (1) *Knowledge Repository Construction*: Problem-solving experiences are distilled into structured knowledge entries. A *structural abstraction* process extracts the problem’s *reasoning structure*, while a *schema generalization* process distills the experience into reusable *solve*, *judge*, and *correct* schemas. (2) *Knowledge Retrieval*: For a new question, its *reasoning structure* is used to retrieve relevant entries. A *diversity reranking* step then selects a final set of entries to provide a robust and strategically diverse context for reasoning.

3.3 Knowledge Repository Construction

To achieve the abstraction described in the previous section, we transform a reflective trace into a structured knowledge entry. This is accomplished through two key processes: structural abstraction and schema generalization.

Specifically, structural abstraction is responsible for generating the discriminating reasoning structure (q_{abs}), while schema generalization is responsible for distilling the generalizable set of schemas ($\{S_{solve}, S_{judge}, S_{correct}\}$). Both processes are orchestrated via dedicated prompts (see Appendix A.3 for templates). For a given problem q and a complete experience E (i.e., the *solve-judge-correct* trajectory), we first utilize a prompt to transform the raw problem q into its *reasoning structure* q_{abs} :

$$q_{abs} \leftarrow \mathcal{M}(p_{abs} \parallel q), \quad (4)$$

Subsequently, we distill the concrete experience E into cognitive schemas. For each stage t in the cognitive process, a dedicated generalization prompt $p_{t\text{-schema}}$ is used to generate the corresponding schema S_t :

$$S_t \leftarrow \mathcal{M}(p_{t\text{-schema}} \parallel q \parallel E), \quad \forall t \in \{\text{solve, judge, correct}\}, \quad (5)$$

The resulting set of schemas, $\{S_{solve}, S_{judge}, S_{correct}\}$, is then paired with the reasoning structure and stored as a single entry in the knowledge repository: $(q_{abs}, \{S_{solve}, S_{judge}, S_{correct}\})$.

To maintain the quality of the knowledge repository, only experiences that meet strict criteria are archived. A *solve schema* (S_{solve}) is created only if the system generates the correct answer directly. A *judge schema* (S_{judge}) is created only when a judgment is accurate (e.g., correctly identifying a flawed solution), thereby building a reliable set of internal critique standards. Finally, a *correct schema* ($S_{correct}$) is stored only if a correction cycle successfully transforms an incorrect solution into a correct one, documenting a reusable recovery strategy.

Before archiving a new knowledge entry, a similarity check is performed against existing entries based on the *reasoning structure* q_{abs} to prevent high redundancy, ensuring the cognitive structure remains both concise and efficient.

3.4 Knowledge Retrieval

As introduced, our framework incorporates a *diversity-aware knowledge retrieval* mechanism. This mechanism is engineered to be discriminating, ensuring the most appropriate knowledge is retrieved

by selecting a set of knowledge entries that balances high relevance with strategic diversity.

The mechanism first establishes relevance. To achieve a precise match that moves beyond superficial phrasing, we apply the *structural abstraction* process to the incoming query q to extract its underlying *reasoning structure*, q_{abs} :

$$q_{\text{abs}} \leftarrow \mathcal{M}(p_{\text{abs}} \parallel q), \quad (6)$$

This abstract representation is then used to perform a similarity search against the entries in the knowledge repository, retrieving an initial pool of j candidates that are highly relevant in their fundamental logic.

From this pool of relevant entries, the mechanism then ensures diversity to select a final, optimal subset of k entries (where $k < j$). A retrieval strategy based solely on similarity risks providing a homogeneous set of examples, which can lead to cognitive fixation [20, 22]. A diverse set, in contrast, provides multiple perspectives, which is crucial for helping the model discriminate between different valid reasoning paths. To achieve this, we employ the Maximal Marginal Relevance (MMR) algorithm [5] to select the final k entries.

Formally, MMR iteratively selects a candidate entry u that maximizes the following objective function:

$$\theta_{\text{MMR}}(x, u, S_{i-1}; \alpha) = \alpha \cdot \theta(x, u) - (1 - \alpha) \cdot \max_{s \in S_{i-1}} \theta(u, s), \quad (7)$$

Here, x is the abstracted query q_{abs} , u is a candidate entry, S_{i-1} is the set of already selected entries, and $\theta(\cdot, \cdot)$ is a similarity function. The first term prioritizes relevance, while the second term penalizes redundancy. The hyperparameter α balances this trade-off.

This process yields a final set of curated knowledge entries. Notably, as each knowledge entry encapsulates a full set of schemas for solving, judging, and correcting, a single retrieval operation efficiently equips the framework with three types of memory required for the entire multi-stage reasoning process. Specifically, the retrieved solve schemas inform the solving phase, the judge schemas guide the judgment phase, and the correct schemas provide exemplars for the correction phase. This curated, phase-specific knowledge context significantly enhances the model’s ability to discriminate between effective and ineffective reasoning strategies, laying a robust foundation for the subsequent reasoning process.

3.5 Knowledge-Augmented Reasoning

The ultimate value of the curated knowledge lies in its application during reasoning. In our framework, the retrieved schemas are seamlessly integrated into each phase of the reasoning engine to guide the model $\mathcal{M}$ towards more efficient and accurate solutions. It is important to note that when the framework is initialized, the knowledge repository is empty. In this “cold start” scenario, the reasoning process defaults to the foundational *solve-judge-correct* cycle described in Section 3.2, operating without any external knowledge.

In the knowledge-augmented *solving phase*, the retrieved *solve schemas* provide proven solution approaches. By prepending this set of schemas, denoted as $\mathcal{S}_{\text{solve}}$, to the prompt, we furnish the model with effective in-context exemplars, increasing the likelihood of generating a correct answer on the first attempt. The solving

step is thus formulated as:

$$(T_0, R_0) \leftarrow \mathcal{M}(p_{\text{solve}} \parallel \mathcal{S}_{\text{solve}} \parallel q), \quad (8)$$

Subsequently, during the knowledge-augmented *judgment phase*, the *judge schemas* contribute standards for accurate evaluation and experience with common pitfalls. Injecting this set of schemas, $\mathcal{S}_{\text{judge}}$, elevates the model’s ability to identify subtle flaws in a candidate solution. The judgment step is enhanced as follows:

$$S_i \leftarrow \mathcal{M}(p_{\text{judge}} \parallel \mathcal{S}_{\text{judge}} \parallel q \parallel (T_i, R_i)), \quad (9)$$

Finally, in the knowledge-augmented *correction phase*, the *correct schemas* offer effective strategies for resolving specific types of errors. Providing this set of schemas, $\mathcal{S}_{\text{correct}}$, guides the model on how to systematically diagnose and fix a flaw, rather than resorting to unguided trial-and-error. The correction step is thereby optimized to:

$$(T_{i+1}, R_{i+1}) \leftarrow \mathcal{M}(p_{\text{correct}} \parallel \mathcal{S}_{\text{correct}} \parallel q \parallel (T_i, R_i) \parallel S_i), \quad (10)$$

This integration of phase-specific knowledge transforms the reasoning loop, driving a solution from an initial candidate to a refined final answer.

A Continually Evolving Cognitive Structure. A key feature of our framework is its ability to learn continuously without parametric updates to the base model $\mathcal{M}$. Inspired by cognitive structure migration theory, the external knowledge repository acts as an evolving *cognitive structure* that assimilates new knowledge, contrasting with traditional fine-tuning, which freezes knowledge into static parameters. This mirrors a human expert, whose core intelligence remains stable while their knowledge structure grows with experience. By applying the *schema generalization* process to distill successful reasoning trajectories into reusable schemas, the system continuously enriches its knowledge repository. This creates a virtuous cycle: improved performance yields higher-quality knowledge schemas, which in turn enhances the cognitive structure for future tasks, enabling perpetual improvement through practice.

4 EXPERIMENTS

In this section, we evaluate Tricog on several widely used complex reasoning datasets. We conduct extensive experiments to demonstrate the effectiveness of our framework by answering the following research questions:

- **RQ1:** How does Tricog perform compared to state-of-the-art (SOTA) baselines across various complex reasoning tasks?
- **RQ2:** What are the contributions of Tricog’s core components and design strategies to its overall performance?
- **RQ3:** How effective is Tricog’s cognitive knowledge in facilitating transfer learning?
- **RQ4:** How does the quality of the knowledge autonomously constructed by Tricog compare to knowledge curated by human experts?

4.1 Experimental Setup

Following established practices for evaluating complex reasoning and self-learning [8, 24], we conduct a comprehensive evaluation of Tricog. Our experiments are performed on a carefully selected suite of benchmarks to assess capabilities across different dimensions.

Table 1: Results (%) of Tricog. The best results are marked in bold, and the second-best results are marked with underline. $\odot$ denotes non-thinking-based, while $\bullet$ denotes thinking-based models. The EL method is only applicable to the EvaLearn dataset, hence the missing values in other benchmarks.

Backbone	Method	AIME		MMLU Pro		EvaLearn						
		20-24	2025	Eng.	Phy.	Sum	Cla	Ex	LR	MR	SR	Overall
$\odot$ DeepSeek-V3	BL	24.8	20.0	72.0	87.6	<u>81.7</u>	72.9	<u>45.2</u>	25.6	71.7	59.6	43.2
	EL	—	—	—	—	76.8	74.7	36.1	24.9	74.8	54.6	41.5
	DC	24.9	16.7	72.8	89.2	78.5	76.9	38.6	26.4	<u>76.4</u>	57.9	43.3
	Xolver	<u>30.1</u>	23.3	<u>73.6</u>	<u>90.4</u>	79.4	<u>78.1</u>	44.5	<u>27.1</u>	<u>73.9</u>	60.5	<u>44.5</u>
	Tricog	38.4	23.3	76.8	91.6	83.9	82.4	48.7	28.7	78.9	63.9	47.2
$\odot$ Qwen3-30B-A3B-Instruct	BL	43.6	<u>33.3</u>	59.6	74.4	58.9	68.1	26.1	12.1	<u>64.7</u>	38.6	28.9
	EL	—	—	—	—	55.3	71.7	22.3	11.6	<u>64.7</u>	35.2	27.9
	DC	42.1	30.0	60.4	75.6	60.7	72.6	<u>26.8</u>	14.9	62.1	37.8	30.7
	Xolver	46.6	30.0	61.6	76.4	<u>61.6</u>	<u>73.6</u>	25.2	16.1	61.3	41.1	31.6
	Tricog	51.9	36.7	63.6	83.6	65.2	74.7	31.9	18.7	66.4	45.4	35.0
$\bullet$ DeepSeek-R1	BL	79.6	<u>70.0</u>	74.4	89.2	86.7	89.6	48.3	<u>41.7</u>	<u>78.3</u>	<u>66.7</u>	<u>55.4</u>
	EL	—	—	—	—	<u>89.3</u>	79.1	<u>48.7</u>	29.6	74.8	51.3	46.4
	DC	73.7	63.3	72.4	84.4	82.1	87.9	47.1	40.6	77.3	60.5	53.5
	Xolver	<u>81.2</u>	<u>70.0</u>	<u>75.2</u>	<u>90.4</u>	88.4	<u>90.1</u>	47.9	39.5	75.6	63.9	53.8
	Tricog	84.2	73.3	76.4	91.2	91.1	92.3	50.4	44.8	80.7	68.9	58.4
$\bullet$ Qwen3-30B-A3B-Thinking	BL	<u>87.9</u>	76.7	71.6	85.2	46.4	62.6	27.7	43.2	<u>66.4</u>	51.3	46.3
	EL	—	—	—	—	48.2	56.1	26.8	41.7	63.5	54.6	45.1
	DC	85.7	73.3	70.8	86.4	44.6	<u>64.8</u>	26.1	45.3	61.3	52.1	47.0
	Xolver	85.7	76.7	<u>73.2</u>	<u>87.6</u>	<u>50.9</u>	62.6	<u>36.9</u>	<u>46.4</u>	62.1	<u>56.3</u>	<u>49.5</u>
	Tricog	90.2	76.7	74.8	88.8	52.7	67.1	46.2	48.1	68.9	59.6	52.7

Datasets. To evaluate complex mathematical reasoning, we employ the AIME [2] benchmark, which is composed of two subsets: 30 questions from the 2025 exam and 133 from the 2020–2024 exams. For professional-level domain knowledge, we use MMLU-Pro [26], comprising 250 multiple-choice questions sampled from each of the *engineering* and *physics* domains. Finally, to directly measure the capacity for continuous learning, we use the sequential benchmark EvaLearn [8], which contains 648 problems organized into 182 sequences.

Baselines. We compare Tricog against a range of baselines representing distinct problem-solving paradigms: **Baseline Prompting (BL)**: A standard, non-iterative prompting approach without any memory or retrieval mechanism, reflecting a one-off inference process. **Dynamic Cheatsheet (DC)** [24]: For each new query, this method retrieves the most similar past interactions and pastes them verbatim into the prompt. **Xolver** [12]: A training-free, multi-agent reasoning framework that equips LLMs with a persistent and evolving global memory, representing a state-of-the-art memory-based agent. **EvaLearn (EL)** [8]: The strong native baseline of the EvaLearn benchmark, which provides all previous problems, solutions, and detailed feedback as context.

Evaluation and Implementation. Our evaluation metrics are tailored to the specifics of each benchmark: we use **Functionally Correct (FC)** for AIME, which verifies the numerical correctness of answers despite formatting variations; **Soft Match (SM)** for MMLU-Pro, a lenient metric that disregards minor punctuation

or whitespace differences; and an **LLM-as-a-judge** protocol for EvaLearn, which uses a powerful LLM to assess nuanced outputs against specific rubrics. We evaluate four frontier LLMs to test the generalizability of our framework: **DeepSeek-R1** [10], **DeepSeek-V3** [16], **Qwen3-30B-A3B-Instruct** [29], and **Qwen3-30B-A3B-Thinking** [29]. For a full breakdown of the datasets, baselines, and implementation specifics, please refer to Appendix A.2 and Appendix A.5.

4.2 Performance Comparison with SOTA (RQ1)

Table 1 summarizes the overall results of Tricog against all baselines across the selected datasets. By analyzing the table, we can draw the following key conclusions:

- **Tricog consistently establishes a new state-of-the-art, demonstrating the superiority of its cognitive architecture.** In nearly all scenarios, Tricog significantly outperforms all baselines. For instance, with DeepSeek-V3 as the backbone on the AIME 2020–2024 benchmark, Tricog achieves an accuracy of 38.4%, surpassing the next-best baseline, Xolver, by a substantial 8.3 percentage points. This superior performance stems from our framework’s ability to build a repository of generalizable knowledge schemas and retrieve them effectively. By matching a problem’s core reasoning structure instead of its surface features, Tricog provides more precise and relevant guidance compared to baselines.

- **Tricog empowers base models to surpass their thinking-specialized counterparts.** A key finding is Tricog’s ability to externalize structured reasoning. For example, the non-thinking DeepSeek-V3, when augmented with Tricog, achieves 91.6% on MMLU Pro (Phy.). This score not only significantly exceeds its own baseline of 87.6% but also surpasses the baseline performance of the more powerful, thinking-specialized DeepSeek-R1 model (89.2%). This demonstrates that the evolving cognitive structure provided by Tricog acts as an effective reasoning scaffold.
- **The performance gap widens with increased experience, confirming Tricog’s capacity for long-term evolution.** The results on the AIME datasets provide clear evidence of our framework’s self-learning capabilities. For DeepSeek-V3, the performance gain of Tricog over its baseline (BL) on the smaller AIME 2025 dataset is 3.3 percentage points. This gain expands dramatically to 13.6 percentage points in the larger AIME 2020–2024 dataset. This scaling effect strongly indicates that as the knowledge repository is enriched with more diverse and reusable schemas, its ability to guide the model on novel problems grows, validating its design as a continuously learning system.
- **Tricog excels on the benchmark specifically designed to test experience-based learning.** The EvaLearn benchmark is explicitly designed to evaluate a model’s ability to learn from past experiences. It is on this benchmark that Tricog demonstrates its most significant and consistent performance gains. For instance, on the EvaLearn Overall score, Tricog delivers substantial improvements for all backbones. This result directly validates our approach, as baselines lacking a structured learning loop treat each problem in isolation, while the solve-judge-correct cycle in Tricog is purpose-built to distill and transfer knowledge.

4.3 Ablation Study (RQ2)

Table 2: Ablation study of Tricog’s components on the AIME 2020–2024 and MMLU-Pro Engineering datasets using DeepSeek-V3 and DeepSeek-R1 models.

Variant	DeepSeek-V3		DeepSeek-R1	
	AIME	MMLU	AIME	MMLU
Tricog (Full)	38.4	76.8	84.2	76.4
w/o Judge & Correct	30.1	72.4	77.4	74.4
w/o Abstraction	32.3	74.0	78.9	75.6
w/o Diversity (MMR)	35.3	75.6	81.9	76.0

To address our second research question (RQ2), we conduct a comprehensive series of ablation experiments to systematically dissect the contributions of each key component within the Tricog framework. We create several variants by removing core designs: the schemas derived from the judgment and correction phases, the structural abstraction process, and the diversity-based retrieval

mechanism. As shown in Table 2, all variants demonstrate a consistent performance degradation compared to the full Tricog framework. The most significant performance decline occurs when the judge and correct schemas are removed, with accuracy on AIME dropping by 8.3% on DeepSeek-V3. This highlights the critical importance of the complete solve-judge-correct cognitive cycle, which enables the system to learn from self-reflection and error correction. Disabling the structural abstraction process also leads to a substantial decrease in performance, validating the effectiveness of matching problems based on their underlying reasoning structure. Finally, removing diversity-based reranking (MMR) results in a smaller yet consistent drop, confirming the value of providing a strategically varied set of knowledge entries to the model.

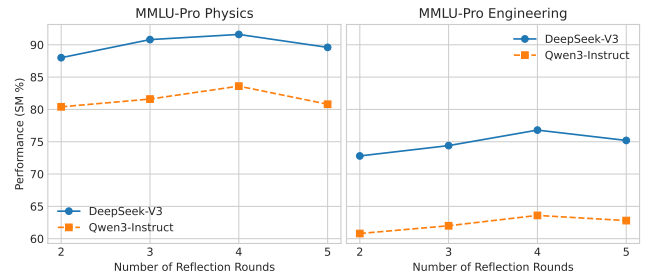


Figure 3: Performance with varying numbers of reflection rounds on the MMLU-Pro Physics and Engineering datasets using DeepSeek-V3 and Qwen3-Instruct models.

We further investigate the impact of the number of iterative judge-correct cycles in the correction phase, with the results shown in Figure 3. The results indicate that additional correction cycles generally lead to performance gains, as they provide more opportunities to identify and rectify errors. However, these gains are not monotonic; performance on both datasets generally peaks at four cycles and then either saturates or slightly decreases. For example, on the MMLU-Pro Engineering dataset, the performance of DeepSeek-V3 improves from two to four cycles. This observation highlights a critical trade-off: while iterative correction is beneficial for refining solutions, excessive cycles can introduce new errors or cause the reasoning to drift from the optimal path. Our findings suggest that an optimal number of correction cycles exists—four in this case—which strikes the best balance between the benefits of refinement and the risk of over-correction.

4.4 Transfer Learning Capability Analysis (RQ3)

To investigate RQ3, which assesses the transferability of the knowledge accumulated in Tricog’s repository, we designed transfer learning experiments. We tested this in both *intra-domain* (from AIME 2020–2024 to AIME 2025) and *cross-domain* (from MMLU Engineering to Physics) scenarios, comparing three setups: a baseline model without access to the repository, Tricog starting with an empty repository on the target task, and Tricog initialized with a pre-populated repository from a source task. The results, depicted in Figure 4, clearly demonstrate significant positive knowledge transfer. For instance, on the AIME 2025 task, the Qwen3-Instruct model’s performance jumps from a baseline of 33.3% to 46.7% when

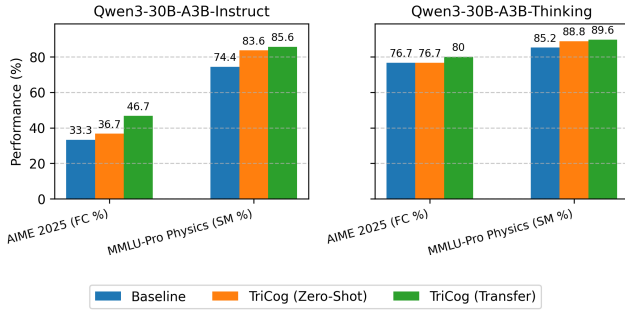


Figure 4: Transfer learning performance on the AIME 2025 and MMLU-Pro Physics datasets using Qwen3-30B-A3B-Instruct and Qwen3-30B-A3B-Thinking models.

initialized with the pre-populated repository. Furthermore, the variant with the transferred knowledge consistently surpasses the one starting from scratch (46.7% vs. 36.7%), highlighting the value of a mature knowledge repository enriched with a diverse set of schemas.

4.5 Memory Quality Analysis (RQ4)

Table 3: Performance comparison between self-learned and human-curated memory on the MMLU-Pro Engineering and Physics datasets using the DeepSeek-V3 model.

Dataset	Variant	Score (SM %)
MMLU-Pro Eng.	No Memory	72.0
	Human-Curated	74.4
	Self-Learned	76.8
MMLU-Pro Phy.	No Memory	87.6
	Human-Curated	89.2
	Self-Learned	91.6

To evaluate the quality of the knowledge autonomously constructed by Tricog (RQ4), we compare it against a knowledge base curated by human experts, following the methodology of prior work [17, 34]. In this setup, the human-curated knowledge consists of a set of high-quality, reusable templates, which form a static knowledge base. As shown in Table 3, both knowledge-augmented approaches deliver substantial improvements over the knowledge-less baseline; for instance, the autonomously constructed knowledge provides a 4.8 percentage point gain on the Engineering dataset, reaffirming the fundamental efficacy of knowledge augmentation. The most striking finding, however, is that the knowledge repository built by Tricog consistently outperforms the one curated by human experts in all settings. Specifically, the autonomously built repository achieves a score 2.4 percentage points higher than the human-curated knowledge on both datasets. This result provides compelling evidence that Tricog’s automated process for schema generalization and knowledge archival is not only highly effective but can also distill reusable knowledge that is even more potent than what is manually selected by humans.

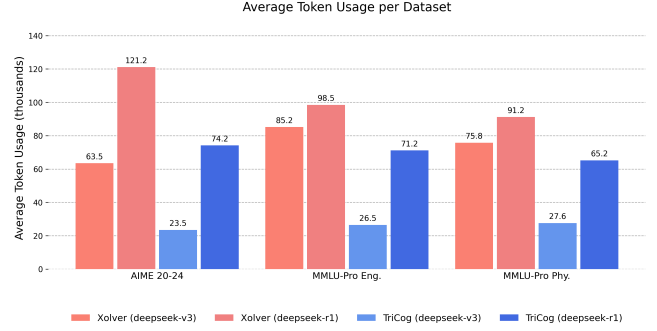


Figure 5: Average token usage of Tricog (TriCog) and Xolver on the AIME 20-24, MMLU-Pro Engineering, and MMLU-Pro Physics datasets using deepseek-v3 and deepseek-r1 models.

In addition to the quality of the constructed knowledge, we analyze the efficiency of our framework, with results presented in Figure 5. The chart clearly demonstrates that Tricog is substantially more token-efficient than Xolver across all datasets and both backbone models. This advantage is particularly pronounced on the AIME 20-24 dataset, where Tricog with DeepSeek-V3 consumes only 23.5k tokens—a reduction of over 60% compared to Xolver’s 63.5k. This significant efficiency gain stems directly from our core design. Through the *schema generalization* process, Tricog distills verbose problem-solving trajectories into concise, high-signal schemas. Unlike methods that rely on entire problem-solution pairs as context, our approach provides the model with more refined and targeted guidance.

5 CONCLUSION

In this paper, we introduce Tricog, a self-learning framework that equips Large Language Models with an external cognitive structure to conduct complex reasoning. Our method involves capturing complete problem-solving experiences through a deep self-reflection cycle (solve, judge, correct), generalizing these experiences into reusable, structured schemas for knowledge transfer, and employing a diversity-aware retrieval mechanism based on a problem’s core reasoning structure. This multifaceted approach not only improves the reasoning accuracy of LLMs but also enables them to learn and evolve from practice without retraining. Extensive experiments demonstrate the superiority of Tricog, showcasing the potential to build more autonomous and adaptive reasoning systems that learn from the past. However, a current limitation lies in the computational resources required for the deep self-reflection cycle and knowledge construction. Future work will therefore focus on optimizing the efficiency of these processes, potentially through more selective criteria for experience archival.

Acknowledgments

This work was partially supported by National Natural Science Foundation of China (Nos. U25B2047, U23A20296, 62272469, 62302513), and The Science and Technology Innovation Program of Hunan Province (No. 2023RC1007).

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] AIME. 2025. AIME Problems and Solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions
- [3] David Paul Ausubel. 2012. *The acquisition and retention of knowledge: A cognitive view*. Springer Science & Business Media.
- [4] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*. PMLR, 2206–2240.
- [5] Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval*, 335–336.
- [6] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshray Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413* (2025).
- [7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
- [8] Shihan Dou, Ming Zhang, Chenhao Huang, Jiayi Chen, Feng Chen, Shichun Liu, Yan Liu, Chenxiao Liu, Cheng Zhong, Zongzhang Zhang, et al. 2025. Evalearn: quantifying the learning capability and efficiency of LLMs via sequential problem solving. *arXiv preprint arXiv:2506.02672* (2025).
- [9] Tao Feng, Pengrui Han, Guanyu Lin, Ge Liu, and Jiaxuan You. [n. d.]. Thought-Retriever: Don't Just Retrieve Raw Data, Retrieve Thoughts. In *ICLR 2024 Workshop: How Far Are We From AGI*.
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyi Zhang, Runxin Xu, Qihao Zhu, Shitong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [11] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. [n. d.]. Measuring Mathematical Problem Solving With the MATH Dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.
- [12] Md Tanzib Hosain, Salman Rahman, Md Kishor Morol, and Md Rizwan Parvez. 2025. Xolver: Multi-Agent Reasoning with Holistic Experience Learning Just Like an Olympiad Team. *arXiv preprint arXiv:2506.14234* (2025).
- [13] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [14] Janak Kapuriya, Mani Kaushik, Debasis Ganguly, and Sumit Bhatia. 2025. Exploring the role of diversity in example selection for in-context learning. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2962–2966.
- [15] Angeliki Lazaridou, Elena Gribovskaya, Wojciech Stokowiec, and Nikolai Grigorev. 2022. Internet-augmented language models through few-shot prompting for open-domain question answering. *arXiv preprint arXiv:2203.05115* (2022).
- [16] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [17] Cong Liu, Jie Wu, Weigang Wu, Xu Chen, Liang Lin, and Wei-Shi Zheng. 2025. Chain of Methodologies: Scaling Test Time Computation without Training. *arXiv preprint arXiv:2506.06982* (2025).
- [18] Aman Madaan, Niket Tandon, Peter Clark, and Yiming Yang. 2022. Memory-assisted prompt editing to improve GPT-3 after deployment. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 2833–2861.
- [19] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2023), 46534–46594.
- [20] David Maxwell, Leif Azzopardi, and Yashar Moshfeghi. 2019. The impact of result diversification on search behaviour and performance. *Information Retrieval Journal* 22, 5 (2019), 422–446.
- [21] Amal Rannen-Triki, Jorg Bornschein, Razvan Pascanu, Marcus Hutter, Andras György, Alexandre Galashov, Yee Whye Teh, and Michalis K Titsias. 2024. Revisiting Dynamic Evaluation: Online Adaptation for Large Language Models. *arXiv preprint arXiv:2403.01518* (2024).
- [22] Rodrygo LT Santos, Craig Macdonald, and Iadh Ounis. 2010. Exploiting query reformulations for web search result diversification. In *Proceedings of the 19th international conference on World wide web*. 881–890.
- [23] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 8634–8652.
- [24] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2025. Dynamic cheatsheet: Test-time learning with adaptive memory. *arXiv preprint arXiv:2504.07952* (2025).
- [25] Xiaoyu Tan, Bin Li, Xihe Qiu, Chao Qu, Wei Chu, Yinghui Xu, and Yuan Qi. 2025. Meta-Agent-Workflow: Streamlining Tool Usage in LLMs through Workflow Construction, Retrieval, and Refinement. In *Companion Proceedings of the ACM on Web Conference 2025*. 458–467.
- [26] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhramil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. 2024. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems* 37 (2024), 95266–95290.
- [27] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent workflow memory. *arXiv preprint arXiv:2409.07429* (2024).
- [28] Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming Tang, and Yong Liu. 2025. From human memory to ai memory: A survey on memory mechanisms in the era of llms. *arXiv preprint arXiv:2504.15965* (2025).
- [29] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [30] Ling Yang, Zhaochen Yu, Tianjun Zhang, Shiyi Cao, Minkai Xu, Wentao Zhang, Joseph E Gonzalez, and Bin Cui. 2024. Buffer of thoughts: Thought-augmented reasoning with large language models. *Advances in Neural Information Processing Systems* 37 (2024), 113519–113544.
- [31] Tianjun Zhang, Shishir G Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E Gonzalez. [n. d.]. RAFT: Adapting Language Model to Domain Specific RAG. In *First Conference on Language Modeling*.
- [32] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).
- [33] Zeyu Zhang, Quanyu Dai, Xiaohu Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems* 43, 6 (2025), 1–47.
- [34] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19632–19642.
- [35] Runhao Zhao, Weixin Zeng, Jiuyang Tang, Yawen Li, Guanhua Ye, Junping Du, and Xiang Zhao. 2025. Towards Unsupervised Entity Alignment for Highly Heterogeneous Knowledge Graphs. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 3792–3806.
- [36] Zexuan Zhong, Tao Lei, and Danqi Chen. 2022. Training Language Models with Memory Augmentation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 5657–5673.
- [37] Mingjun Zhou, Jiuyang Tang, Weixin Zeng, and Xiang Zhao. 2026. INKER: Adaptive dynamic retrieval augmented generation with internal-external knowledge integration. *Information Processing & Management* 63, 3 (2026), 104534.
- [38] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. 2025. MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents. *arXiv preprint arXiv:2506.15841* (2025).

A APPENDIX

A.1 Experimental Environment

All our experiments were orchestrated on a workstation equipped with a single NVIDIA RTX 5090 GPU. The core large language model inferences were performed via API services provided by SiliconFlow. Our experimental environment and the configuration of key components are detailed in Table 4.

Table 4: Experimental environment and key components.

Component	Configuration
API Service	SiliconFlow
Vector Database	FAISS [13]
Embedding Model	Qwen3-Embedding-8B [32]
Embedding Dim.	1024

A.2 Hyperparameters

The key hyperparameters for our framework were kept consistent across all experiments. These values were chosen to balance performance and efficiency. We retrieve an initial pool of $j = 6$ candidates to ensure a sufficiently rich set of perspectives is captured for the diversity reranking stage. From this pool, we select a final set of $k = 3$ memories, a number chosen to provide diverse context without cognitively overloading the LLM with an overly long prompt. The MMR factor of $\alpha = 0.7$ is set to prioritize high relevance while still introducing enough diversity to prevent cognitive fixation. Finally, a redundancy threshold of 0.9 is used to avoid storing near-duplicate entries, thus maintaining a lean and efficient memory base. The specific settings are provided in Table 5.

Table 5: Key hyperparameter settings for our framework.

Hyperparameter	Value	Description
Similarity Retrieval (j)	6	Num. of initial candidates.
Diversity Reranking (k)	3	Num. of final memories.
MMR Factor (α)	0.7	Balances relevance/diversity.
Redundancy Threshold	0.9	Similarity threshold.

A.3 Prompt Templates

The following prompt templates orchestrate our framework and are organized into two categories: **Knowledge Repository Construction**, for creating knowledge entries that pair a problem’s *reasoning structure* with its corresponding *schemas*, and **Knowledge-Augmented Reasoning**, for guiding the *solve-judge-correct* cycle with retrieved schemas. Placeholders in typewriter font (e.g., `question`) are replaced with specific content during runtime.

Knowledge Repository Construction Prompts

Prompt for Structural Abstraction (p_{abs}):

Analyze the following user query and extract its core reasoning structure. Be concise and structured.

- Key variables and values
- Problem constraints and objectives

Keep each section brief and focused. **User Query:** {question}

Prompts for Schema Generalization:

Generated Solve Schema: You are an expert in problem-solving methodologies. Analyze the following solution path and distill the core, generalizable methodology.

Original Problem: {problem}

Solution Path: {solution}

Generated Judge Schema:

You are an expert in evaluation. Analyze the following judgment path and distill the core, generalizable evaluation criteria.

Original Problem: {problem}

Judgment Path: {judgment}

Generated Correct Schema:

You are an expert in error correction. Analyze the following correction path and distill the core, generalizable correction strategy.

Original Problem: {problem}

Correction Path: {correction}

Knowledge-Augmented Solve Prompt (p_{solve})

Solve this math problem step by step.

ORIGINAL PROBLEM: {question}

RETRIEVED SOLVE SCHEMAS (follow this methodology):

IMPORTANT: Use the methodology from the retrieved schemas above. Follow the same problem-solving pattern and approach. Show all steps clearly and wrap your final answer in boxed{{answer}} block.

Knowledge-Augmented Judge Prompt (p_{judge})

Judge this problem solution.

ORIGINAL PROBLEM: {question}

CANDIDATE RESPONSE TO JUDGE:

RETRIEVED JUDGE SCHEMAS (use these criteria):

IMPORTANT: Use the evaluation criteria from the retrieved schemas above. Follow the same evaluation pattern and approach. Produce a score labeled "Score" (1 if correct, 0 if incorrect) with clear reasoning for your judgment.

Knowledge-Augmented Correct Prompt (p_{correct})

You are an expert tutor. Your task is to identify and correct errors in the given solution.

ORIGINAL PROBLEM: {question}

INCORRECT SOLUTION TO CORRECT:

RETRIEVED CORRECT SCHEMAS (follow this strategy):

IMPORTANT: Use the correction strategy from the retrieved schemas above. Follow the same correction pattern and approach. Provide a complete, corrected solution that addresses all identified errors and wrap your final answer in boxed{{answer}} block.

For the EvaLearn [8] benchmark, we adopt its official LL-M-based judge prompt to adhere to the established evaluation protocol and ensure a fair comparison. The prompt is detailed below.

LLM-based Evaluation Prompt

You must strictly follow the guidelines below throughout the grading process; this is very important. Before you begin, please note two points:

You have only two score levels: 0 points and 1 point. 0 points means the student's answer fails to meet any of the requirements in the Rubric. Every requirement in the Rubric is equally important—if the student's answer misses even one requirement, assign 0 points. 1 point means the student's answer fully satisfies all the requirements.

When you are ready to grade, remain calm and focused. Analyze and think through the question step by step, following these steps: First, carefully read and understand each requirement in the Rubric.

Second, examine whether the student's answer fully complies with every requirement, comparing them one by one.

Third, do not rush to your conclusion. Before outputting your final analysis, please review and correct your reasoning. Check that your "Scoring Rationale" refers to all requirements, and that you haven't skipped any because they seemed unimportant. Finally, when you are certain, provide your score and present it in a code block formatted as JSON. Adhere exactly to this output format.

Your output format should be:

[Scoring Rationale]:

[Score]: x points

[JSON]: {"answer_score": score}

Below is the formal question to grade:

<Standard Answer>: {rubric_zh}

<Student Answer>: {solution}

A.4 Case Study

Figure 6 details a case study on an AIME problem, illustrating how Tricog leverages its retrieved knowledge schemas to navigate the full solve-judge-correct cycle, progressing from an initial flawed solution to a final correct answer. Figure 7 presents an example of a top-ranked schema for each phase (solve, judge, and correct), summarized to highlight its core, actionable strategies.

Case Study: A Complex Symmetric Polynomial Problem

Question: Let a, b, c , and d be real numbers that satisfy the system of equations

$$\begin{aligned} a + b &= -3, \\ ab + bc + ca &= -4, \\ abc + bcd + cda + dab &= 14, \\ abcd &= 30. \end{aligned}$$

There exist relatively prime positive integers m and n such that $a^2 + b^2 + c^2 + d^2 = \frac{m}{n}$. Find $m + n$.

0. Memory Retrieval and Reranking.

- **Structural Abstraction:** The system first transforms the question into its core Reasoning Structure: "Solving a 4-variable symmetric polynomial system from partial sums."
- **Retrieval & Reranking:** A two-stage retrieval process is performed independently for each of the three memory types, resulting in three curated sets of memories: M_{gen} (3 schemas), M_{judge} (3 schemas), and M_{correct} (3 schemas).

1. Memory-Augmented Solve Phase. The system uses the curated set of $k = 3$ schemas from the Solve Memory (M_{gen}).

- **Retrieved Schema (Summary):**
 - A primary schema suggesting a direct-solving approach for variables.
 - A secondary schema noting this approach can be complex.

The model follows the primary schema, which leads it down a path that results in an irrational intermediate expression ($a^2 + b^2 + c^2 + d^2 = \frac{797-70\sqrt{58}}{9}$). Lacking a mechanism to critique the validity of this path, the model confidently proceeds with an invalid simplification by dropping the irrational term. **Initial Solve Output:** The final answer is $m + n = 806$.

2. Memory-Augmented Judge Phase. The flawed solution is passed to the Judge module, which uses the curated set of $k = 3$ criteria from the Judge Memory (M_{judge}).

- **Retrieved Judging Criteria (Summary):**
 - Check if the answer's type matches problem constraints (e.g., rational vs. irrational).
 - Identify errors from suboptimal strategy choices.

Based on these criteria, the Judge module provides clear, negative feedback: **Judgment**

Result: The candidate's derivation includes an irrational term ($\sqrt{58}$) that contradicts the problem's premise. The logical path is therefore flawed, and simply dropping the irrational term is an invalid simplification. **Score: 0**

3. Memory-Augmented Correct Phase. The system enters the Correct phase and uses the curated set of $k = 3$ strategies from the Correct Memory (M_{correct}).

- **Retrieved Correction Strategy (Summary):**
 - When direct solving leads to complex expressions, switch to a higher-level algebraic substitution (e.g., let $u = f(x)$).

Guided by this strategy, the model re-examines the equations and applies the substitution $u = 3c^2 - 4c$, simplifying the problem to $u^2 - 14u - 120 = 0$, which leads to the correct rational solution. **Final Output (from Correct Phase):** ...this yields a valid rational solution $c = -2$, $d = 3/2$, leading to $a^2 + b^2 + c^2 + d^2 = 141/4$. Thus, $m = 141$, $n = 4$, and the final answer is $m + n = 145$.

4. Memory-Augmented Judge Phase (Final Verification). The corrected solution is re-submitted to the Judge module for final verification, using the same retrieved criteria.

- **Retrieved Judging Criteria (Summary):**
 - Check if the answer's type matches problem constraints (e.g., rational vs. irrational).
 - Identify errors from suboptimal strategy choices.

This time, the module confirms the solution's validity: **Judgment Result:** The solution now correctly uses algebraic substitution, the result is rational and accurate, and the final format is compliant. No errors are present. **Score: 1**

5. Memory Consolidation. This successful reasoning trajectory yields high-quality experiences, which are generalized and stored back into their respective memory stores for future use:

- **New Judge Memory:** A schema on how to identify intermediate results that conflict with problem constraints (e.g., expecting a rational number).
- **New Correct Memory:** An effective strategy on using variable substitution (e.g., let $u = f(x)$) to simplify high-degree polynomial equations.

Figure 6: An example of Tricog's reasoning process on an AIME problem.

Case Study: Example of Retrieved Cognitive Schemas

1. Retrieved Solve Schema. This schema provides a general methodology for solving polynomial problems with shared complex roots. Although not a perfect match for the symmetric polynomial structure of the current problem, its core algebraic strategy is adopted by the model.

- **Core Theories:** Utilize the Complex Conjugate Root Theorem and Vieta's Formulas to establish equations.
- **Primary Steps:**
 - **Factorization:** Express the polynomial as a product of its factors using known roots (including conjugates).
 - **Expand & Match:** Expand the factored form and match its coefficients with the original polynomial to create a system of equations for the unknowns.
 - **Solve & Verify:** Solve the system and verify the solution by back-substitution.

2. Retrieved Judge Schema. This schema provides a general set of criteria for evaluating mathematical solutions, which is critical for identifying the logical contradiction in the current flawed answer.

- **Format Check:** Does the answer conform to the final format required by the problem (e.g., a reduced fraction m/n , followed by the calculation of $m+n$)?
- **Logic & Constraint Check:** Is the reasoning coherent? Do intermediate results align with the problem's given constraints (e.g., the problem implies a rational solution, but an irrational term appeared in the derivation)?
- **Common Error Identification:** Check for algebraic mistakes, misapplication of theorems (e.g., using a theorem on a non-applicable object), or unsimplified results.

3. Retrieved Correct Schema. This schema provides a systematic set of strategies for diagnosing and correcting a solution after an error is found, with its core idea being to switch to a more macro-level algebraic perspective.

- **Error Diagnosis:** Trace backward from the incorrect conclusion to locate the step that produced the contradiction (e.g., direct solving led to an unreasonably complex expression).
- **Core Correction Strategy:** When a direct solving path is blocked, re-examine the problem's overall structure. Attempt to reconstruct the solution by leveraging higher-level algebraic relationships (such as the **symmetric sums** in this problem) or by applying a **variable substitution** to simplify the problem.
- **Verification & Consolidation:** Verify the new, corrected solution through re-computation and edge-case testing, and consolidate the lessons learned to prevent future errors.

Figure 7: An example of the retrieved cognitive schemas for the AIME problem.

A.5 Experimental Details

Benchmark Datasets. To comprehensively evaluate Tricog's complex reasoning and self-learning capabilities, we select a diverse suite of challenging benchmark datasets that test mathematical reasoning, professional-level knowledge, and the capacity for continuous learning.

- **AIME Exam Questions (2020-2025) [2]:** The American Invitational Mathematics Examination (AIME) is a prestigious high-school competition featuring complex problems that require deep mathematical reasoning and multi-step problem-solving across algebra, combinatorics, number theory, geometry, and probability. We use subsets from AIME 2025 (30 questions) and AIME 2020–2024 (133 questions).
- **MMLU-Pro (Engineering and Physics) [26]:** This is a professional-level subset of the MMLU benchmark focused on physics and engineering, with all questions in a multiple-choice format. It is designed to test deep domain-specific knowledge. We sampled 250 questions from each of the physics and engineering categories.
- **EvaLearn [8]:** A pioneering benchmark designed to evaluate the learning efficiency of LLMs on challenging tasks. It comprises 648 problems organized into 182 sequences. Crucially, EvaLearn requires models to solve problems sequentially, enabling them to leverage experience from prior solutions. The benchmark covers six core tasks: Summarization (Sum), Classification (Cla), Information Extraction (Ex), Logical Reasoning (LR), Mathematical Reasoning (MR), and Sequential Reasoning (SR). This dataset is ideal for directly assessing Tricog's self-learning loop.

Evaluation Metrics. Following existing works [8, 24], we employ a set of evaluation metrics tailored to the specific characteristics of each benchmark dataset to ensure a fair and comprehensive assessment.

- **Soft Match (SM):** A lenient metric used for the multiple-choice questions in MMLU-Pro. It considers an answer correct if it matches the ground truth after ignoring minor formatting differences, such as punctuation or whitespace.
- **Functionally Correct (FC):** A flexible metric applied to the AIME benchmarks. It evaluates whether the model's output satisfies the task-specific constraints and yields a numerically correct result, even if the exact presentation differs from the reference solution.
- **LLM-as-a-judge:** For the nuanced tasks in EvaLearn, we employ an instance-level rubric combined with an LLM-as-a-judge protocol to assess whether the model's output satisfies the corresponding rubrics. Following standard practice, we use GPT-4o [1] as the judging model and include three-shot demonstrations in the prompt to ensure consistent and reliable evaluation.